Patient Risk Intelligence MCP Platform

An Agentic MCP Layer for Clinical Risk Intelligence (Healthcare)

Product Requirements Document (PRD)

Proof of Concept · Version 1.0 · June 25, 2026

Status: Draft for Review · Companion use case to the Batch Release MCP Platform PRD

Table of Contents

1. Executive Summary

This document defines a proof-of-concept (POC) for an agentic Model Context Protocol (MCP) platform delivering real-time Patient Risk Intelligence. Unlike the companion Batch Release Readmission use case, which answers a retrospective question, this platform gives clinicians a live, explainable, multi-domain risk picture of a patient — deterioration risk, medication/polypharmacy risk, and risk signals buried in clinical narrative — fused from four independently governed data domains.

The platform follows the same factory pattern established in the Batch Release PRD: one hardened MCP server template generates four independent servers, each proposed by an onboarding agent, approved by a human, then built and deployed through a repeatable pipeline. At runtime, a LangGraph agent answers clinical questions by calling all four servers through a security gateway enforcing authentication, authorization, rate limiting, and a HIPAA-aware audit trail.

This use case deliberately closes gaps identified in the Readmission POC: it introduces FHIR R4-shaped outputs, clinical terminology codes (LOINC/RxNorm/SNOMED-CT), a vector database as a core (not optional) data domain, and a purpose-of-access audit field — while carrying forward every hardening control already built (RBAC, self-healing, observability, conformance testing).

2. Problem Statement

Bedside nurses, physicians, and case managers need a unified, real-time view of a patient's risk — not just a retrospective readmission score. Today, the signals that matter are scattered: vitals trends sit in one system, lab/diagnosis history in another, medication and interaction data in a third, and the richest signals — a note mentioning a fall, a family history detail, a subtle change in clinical narrative — are buried in free-text documents nobody has time to re-read on every shift.

This creates three concrete problems:

- Speed: early warning signs are caught late because no single view fuses structured and unstructured signals.
- Explainability: a risk number with no citation to its source signals is not actionable or trustworthy at the bedside.
- Access governance: roles need very different slices of this data (a nurse should not see medication-interaction detail; a case manager needs notes but not raw vitals) and today's systems don't enforce that distinction consistently, nor record *why* PHI was accessed.

3. Proposed Solution

A multi-domain agentic MCP layer where each risk dimension — vitals trends, labs/diagnoses, medications/interactions, and clinical notes — is its own independently governed MCP server. A runtime agent fuses all four into one explainable summary, citing which signal came from which source, when asked a question such as “What is this patient's overall risk picture?”

Each server is generated from the same hardened template used in the Batch Release platform. The connector layer now has two implementations of one shared interface — SQL

(TimescaleDB/Postgres) and Vector (Chroma/Qdrant) — proving the architecture's pluggable-connector claim under genuinely different data shapes, not just three similar relational tables.

All outputs are shaped as FHIR R4 resources using terminology codes (LOINC, RxNorm, SNOMED-CT) carried through from the underlying synthetic data, and deterioration risk is computed using NEWS2 — a published, open clinical scoring method — rather than an invented formula.

4. Business Value

Value Driver	Description
Earlier intervention	Risk signals from structured and unstructured sources are fused into one view instead of discovered separately, hours apart.
Explainability	Every risk statement cites its source (vitals trend, a specific lab, a specific note) — not an opaque score.
Governance	Role-specific access (nurse vs. physician vs. case manager) is enforced per tool, with a recorded purpose of access on every PHI touch.
Reusability	The same hardened template and dual-connector pattern (SQL + Vector) onboard future risk domains without rebuilding access logic.

5. Goals & Scope

5.1 In-Scope (this POC)

- 4 independently deployed MCP servers: vitals_trends, labs_diagnoses, medications_interactions, clinical_notes_search — same hardened template as Batch Release.
- Two connector implementations of one shared ABC: SQL (TimescaleDB/Postgres) and Vector (Chroma/Qdrant).
- Build-time onboarding agent + human approval gate, runtime LangGraph agent — unchanged pattern from Batch Release.
- Two-layer deny-by-default RBAC with a new 3-role matrix (clinical-viewer, physician, case-manager) distinct from the Readmission use case, proving the pattern is genuinely data-driven, not hardcoded.
- FHIR R4-shaped tool outputs; LOINC/RxNorm/SNOMED-CT terminology codes carried through from Synthea-generated synthetic patients.
- NEWS2 deterioration scoring (published, open clinical algorithm).
- Purpose-of-access field added to the audit log; a PHI-access-anomaly Grafana panel.
- All hardening controls carried forward unchanged: ownership proof, explain-denial errors, tiered rate limits, MCP conformance testing, trace propagation, usage/cost log, auto-generated API docs, self-healing chaos demo.

5.2 Out of Scope / Deferred to Production

Item	Why deferred

Drug-interaction checking against a licensed clinical database (e.g. Lexicomp, First Databank)	Those databases are commercially licensed; this POC uses a small, curated open rule set over RxNorm codes instead — illustrative, not clinically authoritative.
True POSIX-style per-file ACLs on the document store	MinIO's policies are prefix/bucket-based, not exact per-file ACL parity with ADLS Gen2 — an honest approximation, stated explicitly rather than overclaimed.
Break-glass emergency access	A real, valuable clinical control — temporary elevated access with extra logging — but not required to prove the architecture at POC scale.
Data retention / purge automation	A HIPAA policy and process requirement, not an architecture-proving feature.
Business Associate Agreement (BAA) and other legal documentation	Entirely out of engineering scope; flagged as existing, not built.
Multi-environment promotion, blueprint versioning, secret rotation	Same deliberate cuts as the Batch Release PRD — carried forward unchanged.

6. End-to-End Architecture & Workflow

Figure 1 shows the complete system across three phases: how a new risk domain is proposed and approved (Phase 1), how a live clinical question is answered across four servers (Phase 2), and the security/reliability/observability controls applied throughout (Phase 3). The structure is identical to the Batch Release PRD's architecture — only the domains, connector types, and a handful of healthcare-specific controls differ.

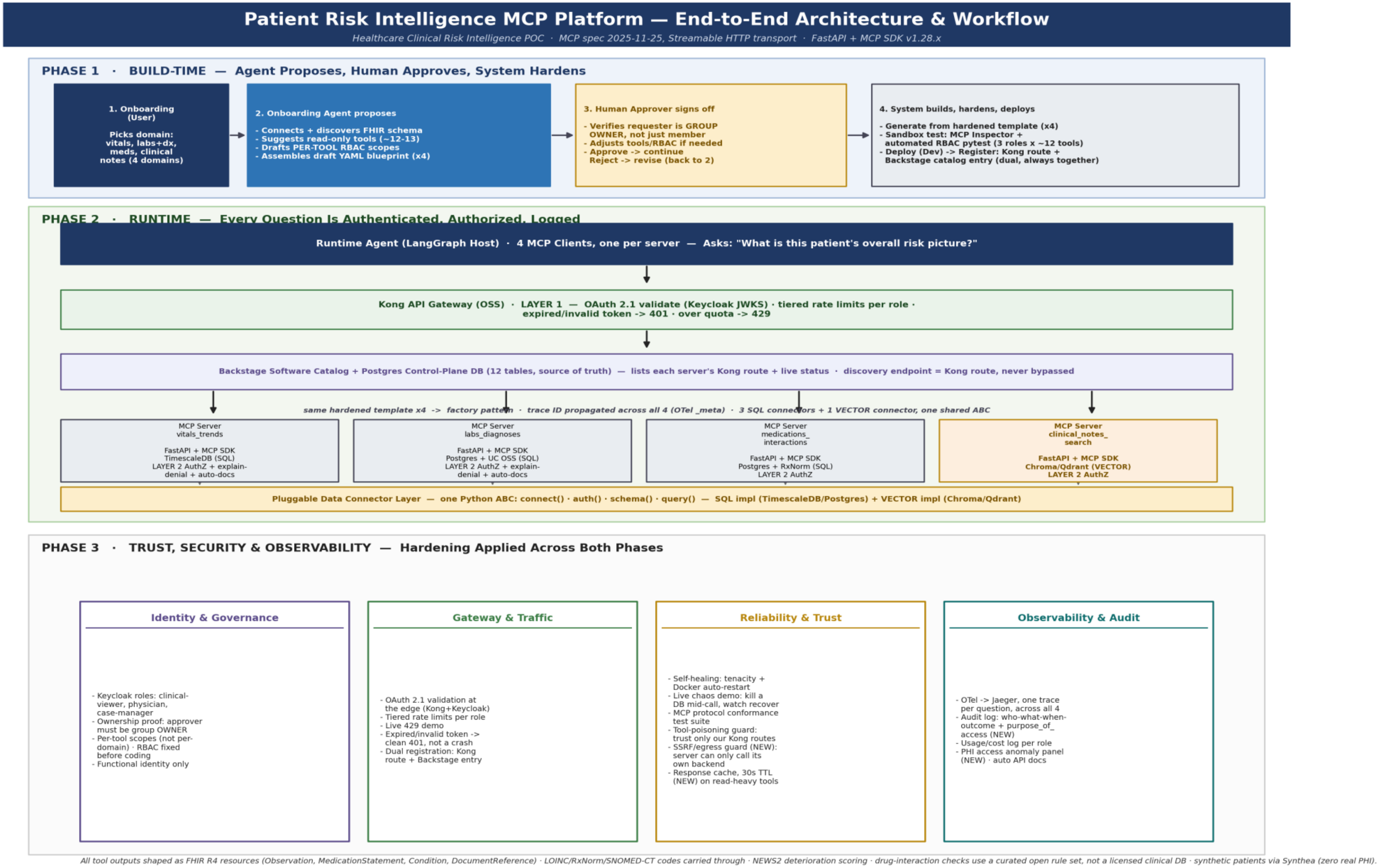

Figure 1 — Patient Risk Intelligence: build-time agent proposal, runtime query path across 4 servers (3 SQL + 1 vector), and the trust/hardening layer applied across both.

6.1 Phase 1 — Build-Time: Agent Proposes, Human Approves, System Hardens

Runs once per risk domain. Unchanged from Batch Release except for scale (4 domains instead of 3) and richer schema discovery (FHIR resource shapes, not just SQL columns).

- Onboarding (User): picks a domain — vitals, labs+diagnoses, medications, or clinical notes.
- Onboarding Agent (proposes): discovers the FHIR-shaped schema, suggests a read-only tool set (~3 tools per domain, ~12-13 total), drafts per-tool RBAC scopes, assembles a draft YAML blueprint.
- Human Approver (governs): verifies group ownership, adjusts tools/RBAC, signs off — or rejects back to the agent.
- System (executes): generates from the hardened template, runs MCP Inspector + automated RBAC pytest (3 roles × ~12 tools), deploys, and registers — Kong route + Backstage catalog entry, dual and always together.

6.2 Phase 2 — Runtime: Every Question Is Authenticated, Authorized, Logged

- Runtime Agent (LangGraph Host): receives the clinical question, attaches the caller's OAuth token, opens 4 MCP Clients — one per server.
- Kong API Gateway (Layer 1): validates the token, applies tiered rate limits per role, routes the request. Invalid/expired → 401; over-quota → 429.
- Backstage Software Catalog: discovery layer listing each server's current Kong route and live status.
- MCP Servers (Layer 2, four independent instances): re-verify the JWT, check scopes per tool, return data or a 403 with an explicit denial reason.
- Pluggable Data Connector Layer: now has two real implementations of the same ABC — SQL (TimescaleDB/Postgres) for three servers, Vector (Chroma/Qdrant) for clinical_notes_search — the strongest proof yet that the architecture is genuinely source-agnostic.

6.3 Phase 3 — Trust, Security & Observability

All controls from the Batch Release PRD are carried forward unchanged, plus four healthcare-specific or previously-missing additions, closed during a gap audit against the Fixed Core specification:

- Audit log gains a purpose_of_access field (e.g. “deterioration review”, “discharge planning”) — HIPAA's accounting-of-disclosures expectation is about why, not just who/what/when.
- A new PHI-access-anomaly Grafana panel flags unusual access patterns (the same patient viewed by many distinct users, access outside expected hours) — built from the existing audit log, no new infrastructure.
- SSRF/egress guard (NEW): each server's connector is hardcoded to its own configured backend at startup — it cannot be redirected to call an arbitrary URL at runtime, closing a gap in the Fixed Core's “tenant isolation · SSRF · egress guard” requirement.

- Response caching, 30s TTL (NEW): read-heavy tools (get_vitals_trend, get_lab_trend) cache results briefly — closing the “rate limiting · caching” half of the Fixed Core that previously only had rate limiting.

Section 9 maps every hardening item — inherited and new — to the exact file that implements it.

7. Detailed Component Description

7.1 Data Layer

Four independent data domains, fed entirely by Synthea — MITRE's open-source synthetic patient generator, which produces realistic FHIR R4 / CSV records with zero real PHI.

Domain	Storage	FHIR resource shape	Terminology
vitals_trends	TimescaleDB	Observation (vital-signs)	LOINC
labs_diagnoses	Postgres + Unity Catalog OSS	Observation (labs), Condition	LOINC, SNOMED-CT, ICD-10
medications_interactions	Postgres	MedicationStatement	RxNorm
clinical_notes_search	Chroma/Qdrant (vector)	DocumentReference	free text, embedded

7.2 Connector Layer

The same Python ABC from Batch Release (connect/auth/schema/query) now has two concrete implementations: a SQL implementation shared by vitals_trends, labs_diagnoses, and medications_interactions; and a Vector implementation for clinical_notes_search, where query() performs a similarity search instead of a SQL filter. This is the architecture's clearest proof that “any source” isn't just a slide claim.

7.2.1 Hardened Template — Fixed Core Components

Every one of the 4 servers inherits the same Fixed Core, unmodifiable by the connector or blueprint. A gap audit against the original Fixed Core specification found two items listed but not yet implemented; both are now closed.

Fixed Core component	Status	Implemented in
Transport / session	Inherited	MCP SDK (Streamable HTTP)
OAuth 2.1 resource server (token+audience)	Inherited	Kong + Keycloak
AuthZ / policy engine — deny-by-default	Inherited	backend/shared/auth.py
Input validation (schemas)	Inherited	Pydantic models per tool
Tool registry · execution engine	Inherited	backend/servers/*/tools.py
Rate limiting	Inherited	Kong tiered rate-limit plugin
Caching	CLOSED (was missing)	backend/shared/cache.py — 30s TTL
Structured errors	Inherited	backend/shared/auth.py (explain-denial)
OpenTelemetry + audit instrumentation	Inherited	backend/shared/telemetry.py, audit.py

Tenant isolation · SSRF · egress guard	CLOSED (was missing)	backend/shared/egress_guard.py

7.3 MCP Servers (4)

Server	Tools	Required scope
vitals_trends	get_vitals_trend, compute_news2_score, list_abnormal_vitals	mcp.vitals.read
labs_diagnoses	get_lab_trend, get_active_diagnoses, get_diagnosis_history	mcp.labs.read
medications_interactions	get_active_medications, check_drug_interactions, get_polypharmacy_risk	mcp.meds.read
clinical_notes_search	semantic_search_notes, get_recent_notes	mcp.notes.read

compute_news2_score implements NEWS2 (NHS's published early-warning score) — a real, citable clinical algorithm, not an invented formula. check_drug_interactions uses a small curated open rule set over RxNorm codes; it is illustrative and explicitly not a substitute for a licensed clinical drug-interaction database.

7.4 Onboarding Agent (Build-Time)

Unchanged in role from Batch Release: one process, internal stages (discover → suggest → draft RBAC → assemble blueprint), output is a YAML file only — it never deploys anything itself.

7.5 Runtime Agent (LangGraph)

The MCP Host, holding 4 MCP Clients (one per server). Synthesizes a single explainable answer that cites which signal came from which server — e.g. “NEWS2 score 6 (vitals_trends); 3 active high-risk medication interactions (medications_interactions); a note from 2 days ago mentions a fall risk concern (clinical_notes_search).”

7.6 API Gateway — Kong

Unchanged from Batch Release — Layer 1 security/traffic control: OAuth 2.1 validation, tiered rate limits per role, routing.

7.7 Identity Provider — Keycloak

Three roles for this use case: clinical-viewer (nurse), physician, case-manager — a deliberately different matrix from the Batch Release roles, proving RBAC is driven by each blueprint, not hardcoded per domain.

7.8 Registry / Catalog — Backstage + Persistent Control-Plane DB

A gap audit found the Batch Release registry (a flat JSON file) under-implements the original specification, which calls for a persistent database as the control-plane source of truth. This PRD closes that gap with a dedicated Postgres database (registry-db) holding 12 logical tables:

- products, mcp_servers, connectors, tool_specs — what exists and how it's built
- rbac_mappings, deployment_environments — who can access what, where
- gateway_routes, api_registry, agent_integrations — how each server is reachable

- audit_events, health_checks, schema_snapshots — what happened and when

Backstage's software catalog continues to provide the human-facing discovery UI; the registry-db is what the onboarding agent, CI/CD pipeline, and self-heal automation actually read and write against.

7.9 Observability & Self-Healing

OpenTelemetry → Jaeger with one trace ID per question across all 4 servers; the audit log's purpose_of_access field and the PHI-access-anomaly Grafana panel. Self-heal automation continuously monitors and repairs known failure conditions:

Failure	Self-heal action
MCP server container down	Restart / redeploy
Kong route missing	Re-register route
Backstage catalog entry missing	Re-register API
Agent tool registry entry broken	Re-sync MCP endpoint
Unity Catalog OSS permission drift	Raise governance ticket
Qdrant collection / MinIO policy drift	Raise data-owner approval workflow
Secret expired (Vault)	Rotate credential
Health check failed	Trigger rollback or restart
Schema drift (FHIR shape changed)	Re-run discovery, generate diff

7.9.1 CI/CD Pipeline & Change Governance

GitHub Actions implements the same pipeline stages as the original specification's CI/CD Agent:

- Validate request → Generate MCP spec → Security scan → Generate infrastructure → Build server image
- Run unit tests → Run sandbox integration tests → Deploy to Dev → Run smoke tests → Promote to Stage
- Change-governance gate → Deploy to Prod → Register gateway/API/agent → Publish documentation

The change-governance gate is the ServiceNow-equivalent from the original specification, implemented as a lightweight Postgres change-request table for POC speed (GLPI, a real open-source ITSM tool, is the documented path if a fuller tool is wanted later). Responsibilities: create a change request for Stage/Prod promotion, attach deployment evidence/test results/security scan results, track the implementation window, update status post-deploy, trigger rollback if failed.

7.9.2 Documentation Automation

For every onboarded MCP server, documentation is auto-generated covering: product overview, business/technical owner, data sources, FHIR resource shapes exposed, exposed MCP tools, tool risk classification, security model, RBAC groups, service account, gateway route, Backstage registration, agent registry integration, deployment history, support process, runbook, troubleshooting guide, known limitations, and change history — the same 20-item structure specified in the original document, generated from the registry-db rather than maintained by hand.

7.10 Frontend Application

Same two-page Next.js app as Batch Release — CopilotKit chat (with human-in-the-loop approval cards) and a Tremor dashboard, now including the PHI-access-anomaly panel alongside the existing KPI/registry/self-healing widgets.

7.11 Backend Services

Four independent MCP server processes, the onboarding agent service, the shared connector library (SQL + Vector implementations), the RBAC/AuthZ middleware, the new egress-guard and cache modules, and the registry API backed by registry-db. See Section 11 for the full repository layout.

8. Security & Governance Model

8.1 RBAC Mapping

A new role/tool matrix, deliberately different from the Batch Release PRD's matrix — demonstrating RBAC is genuinely driven by each blueprint, not hardcoded:

Role	vitals_trends	labs_diagnoses	medications_interactions	clinical_notes_search
clinical-viewer (nurse)	Allow	Allow	Deny	Deny
physician	Allow	Allow	Allow	Allow
case-manager	Deny	Deny	Deny	Allow

8.2 Ownership Proof

Unchanged from Batch Release: the human approver step verifies the requester is a Keycloak group owner, not merely a member, before any blueprint is approved.

8.3 Deny-by-Default Flow

Unchanged two-layer flow: Kong (token validity, rate limit) then MCP server (group/scope intersection per tool). Either failing returns 401/403/429; both passing returns 200 with data.

8.4 Audit Log Schema (extended)

Every call is logged with the same structure as Batch Release, plus one new field:

```
{
  "who": "<oid from token>",
  "what": "<tool name called>",
  "when": "<ISO 8601 timestamp>",
  "outcome": "200 | 401 | 403 | 429",
  "reason": "<populated on 403, e.g. missing scope mcp.meds.read>",
  "purpose_of_access": "<e.g. deterioration review, discharge planning>",
  "trace_id": "<OpenTelemetry trace id>"
}
```

8.5 Explain-Denial Errors

Unchanged: every 403 names the specific missing scope rather than returning a bare denial.

8.6 FHIR & Terminology Standards

All tool outputs are shaped as FHIR R4 resources (Observation, MedicationStatement, Condition, DocumentReference), carrying LOINC codes for vitals/labs, RxNorm codes for medications, and SNOMED-CT/ICD-10 codes for diagnoses — all sourced directly from Synthea's native output rather than invented. Deterioration risk uses NEWS2, a published NHS early-warning score.

8.7 Policy Guardrails

A gap audit found this PRD described guardrails only informally; the specification calls for three explicit guardrail categories, now stated directly:

- Query guardrails: block DROP/DELETE/UPDATE/INSERT/MERGE unless explicitly approved · limit result rows · limit execution time · restrict cross-catalog queries · enforce approved schemas · mask PHI columns (patient name, MRN, DOB).
- File/vector guardrails: limit sample size returned per semantic search · block raw document download unless approved · enforce allowed note types · log every read.
- Agent guardrails: only approved agents may call production MCP servers · tool-invocation approval for high-risk actions · human-in-the-loop for anything touching identifiable PHI · per-agent rate limits · per-product quotas.

9. Hardening & Trust Enhancements

All hardening items from the Batch Release PRD are carried forward unchanged (not repeated in full here — see that PRD's Section 9). This table lists only what is new or changed for Patient Risk Intelligence.

Enhancement	Description	Implemented In
FHIR R4-shaped outputs	Tool responses shaped as Observation/MedicationStatement/Condition/DocumentReference, not raw rows.	backend/shared/fhir_shape.py
Terminology codes carried through	LOINC (vitals/labs), RxNorm (meds), SNOMED-CT/ICD-10 (diagnoses) surfaced in every relevant tool output.	backend/servers/*/tools.py
Vector DB as a core domain	clinical_notes_search is a first-class server, not an optional add-on — proves the connector ABC against a non-SQL source.	backend/connectors/vector_connector.p
Purpose-of-access audit field	Every audit record includes why PHI was accessed, not just who/what/when.	backend/shared/audit.py
PHI access anomaly panel	Grafana panel flagging unusual access patterns from the existing audit log.	frontend/app/dashboard/page.tsx
NEWS2 deterioration scoring	Published, open clinical early-warning algorithm — not an invented risk formula.	backend/servers/vitals_trends/news2.py
Curated drug-interaction rule set	Small open rule set over RxNorm codes, explicitly not a licensed clinical DB substitute.	backend/servers/medications_interactio
Synthetic FHIR patient data	Synthea-generated patients — zero real PHI anywhere in the POC.	infra/synthea/
SSRF / egress guard	Each server's connector is hardcoded to its own backend; cannot be redirected at runtime. Closes a Fixed Core gap.	backend/shared/egress_guard.py
Response caching (30s TTL)	Read-heavy tools cache briefly. Closes the other half of the Fixed Core's “rate limiting · caching” requirement.	backend/shared/cache.py
Persistent control-plane DB	Replaces a flat JSON registry with a real Postgres DB (12 tables) as source of truth, per the original specification.	infra/postgres/init-registry-db.sql
Explicit Policy Guardrails	Query/file/agent guardrails stated directly rather than implied.	backend/shared/auth.py, tools.py

10. Technology Stack & Version Compatibility

MCP protocol and SDK versions are identical to the Batch Release PRD — both platforms must stay on the same versions since they share the hardened template and connector ABC.

Component	Choice	Version / Note
MCP protocol spec	Model Context Protocol	2025-11-25 (current stable release)
MCP SDK (Python)	mcp	v1.28.x stable line — pin mcp>=1.27,<2 (v2 is alpha)
Transport	Streamable HTTP	Current standard for remote MCP servers
Language runtime	Python	3.12 or 3.13
Web framework	FastAPI	0.136.x — requires Pydantic v2
Time-series database	TimescaleDB	Postgres extension — vitals_trends domain
Relational database	PostgreSQL	labs_diagnoses, medications_interactions domains
Data governance	Unity Catalog OSS	Apache 2.0, LF AI & Data — governs labs_diagnoses
Vector database	Chroma or Qdrant (Qdrant recommended — built-in dashboard)	clinical_notes_search domain
Control-plane database	PostgreSQL (registry-db)	12-table source of truth — products, mcp_servers, rbac_mappings, audit_events, etc.
Synthetic patient data	Synthea	MITRE's open-source FHIR R4 patient generator
Clinical scoring	NEWS2	Published NHS early-warning score algorithm
Medication terminology	RxNorm	NLM, free/public
API Gateway	Kong Gateway OSS	3.14 (latest stable)
Identity Provider	Keycloak	26+ recommended
Service catalog	Backstage	CNCF — Azure API Center alternative
Agent orchestration	LangGraph	Runtime agent (Host)
Frontend	Next.js + React + Tailwind + CopilotKit + Tremor	Same stack as Batch Release
Tracing / Metrics	OpenTelemetry → Jaeger / Prometheus + Grafana	—

CI/CD	GitHub Actions	—

Compatibility rule (unchanged from Batch Release): pin all MCP servers — across both PRDs — to the exact same mcp SDK version to avoid protocol-negotiation mismatches.

11. Project Directory Structure

A separate repository, mirroring the Batch Release layout exactly so the same engineers can move between both POCs without relearning structure:

```
patient-risk-intelligence-poc/
├── README.md
├── docker-compose.yml
├── .env.example
│
├── infra/
│   ├── kong/kong.yml                      # Routes, OAuth, tiered rate-limit
│   ├── keycloak/realm-export.json          # clinical-viewer/physician/case-mgr
│   ├── synthea/                             # Synthetic FHIR patient generator config
│   └── postgres/
│       ├── init-labs-diagnoses.sql
│       ├── init-medications.sql
│       ├── init-timescale-vitals.sql
│       └── init-registry-db.sql            # NEW — 12-table control-plane schema
│
├── backend/
│   ├── shared/
│   │   ├── connector_base.py              # Pluggable Connector ABC
│   │   ├── fhir_shape.py                   # Shapes tool output as FHIR R4
│   │   ├── auth.py                         # JWT decode + RBAC engine (Layer 2)
│   │   ├── audit.py                        # Audit log + purpose_of_access
│   │   ├── telemetry.py                    # OpenTelemetry + trace propagation
│   │   ├── tool_trust.py                   # Tool-poisoning guard
│   │   ├── usage_log.py                    # Usage/cost log per role
│   │   ├── egress_guard.py                 # NEW — SSRF/egress guard (Fixed Core)
│   │   └── cache.py                        # NEW — 30s TTL response cache (Fixed Core)
│   ├── connectors/
│   │   ├── sql_connector.py                # TimescaleDB/Postgres implementation
│   │   └── vector_connector.py             # NEW — Chroma/Qdrant implementation
│   ├── servers/
│   │   ├── vitals_trends/
│   │   │   ├── main.py                     # FastAPI + MCP SDK app
│   │   │   ├── news2.py                     # NEW — NEWS2 scoring
│   │   │   ├── tools.py
│   │   │   ├── blueprint.yaml
│   │   │   └── Dockerfile
│   │   ├── labs_diagnoses/         (same structure)
│   │   ├── medications_interactions/
│   │   │   ├── interactions.py             # NEW — curated RxNorm rule set
│   │   │   └── ...
│   │   └── clinical_notes_search/
│   │       ├── main.py                     # Vector connector, not SQL
│   │       └── ...
│   ├── onboarding_agent/                   # discover.py, suggest_tools.py, ...
│   ├── registry/                           # Backstage catalog sync + registry-db API
│   └── tests/
│       ├── test_rbac_matrix.py             # 3 roles x ~12 tools
│       ├── test_self_healing.py
│       └── test_mcp_conformance.py
│
├── agent/
│   └── runtime_agent.py                    # LangGraph Host, 4 MCP Clients
│
├── frontend/
│   ├── app/chat/page.tsx                   # CopilotKit chat + approval card
```

```
│   ├── app/dashboard/page.tsx             # Tremor + PHI anomaly panel (NEW)
│   └── components/
│
├── .github/workflows/ci.yml
└── docs/
    ├── patient_risk_architecture_workflow.png
    └── prd.docx
```

Files marked NEW above (egress_guard.py, cache.py, init-registry-db.sql) were added during a gap audit against the original Fixed Core specification — they did not exist in the first draft of this PRD. fhir_shape.py, audit.py, and tool_trust.py existed already but are listed without the NEW marker since they were correctly included from the start.

12. Success Criteria / Demo Acceptance Checklist

- A physician asks the risk question and receives a correct, synthesized, FHIR-grounded answer citing all 4 servers.
- A clinical-viewer (nurse) asking the same question is correctly denied medications_interactions and clinical_notes_search, with explicit denial reasons.
- compute_news2_score returns a correct NEWS2 score matching the published algorithm for a known synthetic vitals set.
- semantic_search_notes returns relevant notes for a free-text query, demonstrating the vector connector works against the same ABC as the SQL connectors.
- Every audit record includes a populated purpose_of_access field.
- The PHI-access-anomaly panel flags a deliberately triggered unusual-access pattern during the demo.
- Rate-limit (429), expired-token (401), and self-healing chaos demos all reproduce successfully (carried forward from Batch Release).
- The automated RBAC pytest suite passes all 3×12 role/tool combinations.
- A deliberate attempt to make a server call an arbitrary external URL is blocked by the egress guard (SSRF gap closed).
- A repeated identical call to get_vitals_trend within 30s is served from cache, visible as a faster response/trace span.
- The registry-db (not a flat file) is queried live to answer “which servers are registered and what's their Kong route” during the demo.

13. Risks & Assumptions

Risk / Assumption	Mitigation
The curated drug-interaction rule set is not clinically authoritative	Explicitly documented as illustrative; production would require a licensed database (Lexicomp, First Databank).
MinIO does not give exact ADLS-style per-file ACLs	Stated explicitly as an honest approximation, not overclaimed as equivalent.
Vector search relevance depends on embedding quality	Acceptable for a POC; embedding model choice documented as a tunable, not fixed.
All patient data is synthetic (Synthea)	By design — zero real PHI risk; results illustrate the pattern, not clinical accuracy on real patients.
Two PRDs (Batch Release, Patient Risk) must stay on matching MCP SDK/template versions	Both reference the same Section 10 version table; any upgrade must be applied to both repos together.

14. Appendix

Appendix A — Glossary

Term	Definition
FHIR R4	Fast Healthcare Interoperability Resources, release 4 — the HL7 standard for exchanging healthcare data.
LOINC	Logical Observation Identifiers Names and Codes — standard codes for lab/vitals observations.
RxNorm	NLM's standardized medication naming/coding system.
SNOMED-CT	A comprehensive clinical terminology used for diagnoses and conditions.
NEWS2	National Early Warning Score 2 — NHS's published deterioration-risk algorithm.
Synthea	MITRE's open-source synthetic patient data generator (FHIR R4 native).

Appendix B — Audit Log Example Record


```
{ "who": "dr.patel@example.com", "what": "check_drug_interactions", "when": "2026-06-
25T14:02:11Z", "outcome": "200", "purpose_of_access": "medication reconciliation", "trace_id":
"7ac1..." }
```


Appendix C — Document History

Version	Date	Change
1.0	2026-06-25	Initial draft — companion to Batch Release MCP Platform PRD v1.0
1.1	2026-06-25	Gap audit: closed SSRF/egress guard, response caching, persistent control-plane DB, explicit policy guardrails, CI/CD + change governance, documentation automation, self-heal failure table

Appendix D — Request to Production, Step by Step

- 1. Product owner opens onboarding UI
- 2. Selects a risk domain (vitals / labs+diagnoses / medications / clinical notes)
- 3. Provides backend connection details (TimescaleDB/Postgres connection or Qdrant collection)
- 4. Selects criticality tier and PHI data classification
- 5. Platform validates ownership and RBAC
- 6. Onboarding agent scans schema/FHIR shape and permissions
- 7. Onboarding agent suggests MCP tools
- 8. Product owner reviews and approves tools/RBAC
- 9. MCP server spec (YAML blueprint) is generated
- 10. Hardened server template is instantiated (Fixed Core, including egress guard + cache)
- 11. Sandbox tests are executed (15-item checklist, including PHI leakage and rollback tests)
- 12. Dev deployment is completed

- 13. Stage deployment runs integration/security tests
- 14. Change request is created for Prod (lightweight simulator / GLPI)
- 15. Prod deployment is approved and released
- 16. Kong route is registered
- 17. Backstage catalog entry is registered; registry-db updated
- 18. Agent tool registry integration is completed
- 19. Documentation is auto-generated
- 20. Monitoring and self-heal are enabled